MESA: Emergent Misalignment in Multi-Agent AI Systems under Resource Scarcity

An agent-based simulation study in NetLogo

Richie Mouhouadi
MSc Data Science & AI, DSTI School of Engineering (Data ScienceTech Institute)
October 2026

Abstract

Multi-agent AI systems, in which an orchestrator delegates subtasks to worker agents, are increasingly deployed on shared and limited computing resources. Alignment research mostly evaluates agents one at a time. This paper asks whether a population of individually aligned agents can still produce misaligned behaviour at the system level when resources are scarce. I present MESA (Multi-Agent Emergent Systems Arena), an agent-based model documented with the ODD protocol and implemented in NetLogo 7.0.4. Workers follow fixed, honest rules; under resource pressure their progress reports become noisier and their logging less reliable, and the orchestrator allocates resources by trust. Five metrics measure misalignment: information asymmetry (IAI), resource inequality (Gini), task completion drift (TCD), oversight erosion (OER) and cascade failure (CFP).

Across 4,080 simulation runs in five experiments, misalignment grows smoothly and stays low up to a scarcity of 0.90, then the system tips: at scarcity 0.95 most workers die and runs end early. Topology matters only at that edge, where a two-tier hierarchy collapses in 83% of runs against 43% for a star and 27% for a ring ($\chi^2 = 20.4$, $p < 0.001$). The location of the tipping point is set by the survival threshold. An ablation shows that information asymmetry emerges without the model's only under-reporting rule, which adds to it only above scarcity 0.80. A passive monitor tracks the information gap closely ($r = 0.92$) but almost never raises its alarm. I discuss what these results suggest, with due caution, for people running several AI agents on a personal computer, and propose a validation experiment with real agents on a 4 GB laptop.

In plain words

Imagine a team of AI assistants working for one AI manager that shares out a limited budget of computing resources. Nobody in the team is programmed to lie or to grab more than its share. When the budget gets tighter, things get slowly worse: stretched assistants make sloppier reports and skip writing things down, so the manager's picture of the work drifts away from reality. Past a certain point the system does not bend any more, it breaks, with little warning. How the team is organised matters only at that breaking point, where a team with middle managers fails most often. Dishonesty is not what causes the problem, and the watchdog meant to catch it rarely sounds the alarm.

1 Introduction

AI agents are moving from single assistants to teams. In current orchestration frameworks, a coordinating agent splits a goal into subtasks, hands them to worker agents, and assembles their results. These systems run on shared, finite resources: context windows, token budgets, rate limits, memory and processor time. When many agents compete for them, each one works with less.

Most alignment work evaluates the behaviour of one model at a time, for example whether an agent acts against its operator's interests in a given scenario (Lynch et al., 2025). Far less is known about what happens when many agents, each behaving acceptably on its own, interact under constraints. Complex-

systems research suggests that collective behaviour can differ qualitatively from individual behaviour, and that such systems can shift abruptly from one regime to another rather than degrading smoothly (Scheffer et al., 2009).

MESA was built to study this question in a controlled setting. Its central question is: **under what conditions does a population of individually aligned AI agents produce collectively misaligned behaviour at the system level?** The model was first specified as an ODD protocol (Grimm et al., 2020) for the Agent-Based Modelling course of the DSTI MSc. This paper extends that specification with an implementation, a systematic experimental campaign, an evaluation of the patterns the ODD predicted, and a proposal to test the findings with real agents.

The paper makes four contributions: (1) a documented, runnable model of a resource-constrained orchestrator–worker system with five misalignment metrics; (2) results from 4,080 runs covering scarcity, topology, survival threshold, population size and an ablation of the only biased rule; (3) an honest evaluation of which predicted patterns were and were not reproduced; and (4) practical implications and a validation protocol for real multi-agent setups on consumer hardware, including one built on a production orchestration platform.

2 Background

Emergence and critical transitions. Complex systems often show regime shifts: a slowly changing driver produces little visible change until a threshold is crossed, after which the system moves rapidly to a different state (Scheffer et al., 2009). Networks differ in how they absorb node failures: structures in which many nodes depend on a few intermediaries are vulnerable when those intermediaries fail (Barabási, 2016).

Multi-agent systems and alignment. Classical multi-agent systems research describes agents through perception, deliberation and action (Shoham & Leyton-Brown, 2009). Alignment research asks how to make AI systems act in accordance with human values and intentions (Gabriel, 2020). Simulations with language-model agents have shown that believable collective behaviour can arise from simple individual architectures (Park et al., 2023), and work on oversight has argued that visibility into agent activity, through identifiers, logging and monitoring, is a precondition for governing deployed agents (Chan et al., 2024). MESA connects these strands: it treats logging and reporting as resource-consuming activities and asks what happens to oversight when resources run short.

A production analogue. MESA's architecture mirrors current durable orchestration platforms. Mistral Workflows, for example, runs multi-step AI processes on the Temporal durable-execution engine: a workflow coordinates activities that call models and tools, every step is written to an event history, and OpenTelemetry traces are produced without extra setup (Mistral AI, 2026). Table 1 maps MESA's entities onto that architecture. The mapping is what makes the model's metrics observable in a real system, and it is the basis of the validation experiment proposed in Section 8.

Table 1. MESA entities and their production counterparts.

MESA	Mistral Workflows / Temporal	Observable signal
Orchestrator	Workflow	Workflow state and event history
Worker agents	Activities (model and tool calls)	Activity results and retries
Resource pool	Token budgets, context limits, rate limits	Tokens consumed, throttled calls
Oversight log	Event history, OpenTelemetry traces	Share of steps traced and logged
Monitor	Tracing or alerting sidecar	Alerts raised

Agent-based modelling. Agent-based models (ABMs) are well suited to this question because the property of interest is not written into any agent's rules and is difficult to derive analytically. ABMs allow

controlled parameter sweeps with many stochastic replicates, which is what a causal claim about scarcity or topology requires.

3 The MESA model

This section summarises the model as implemented (version 2). The full description follows the ODD protocol (Grimm et al., 2020); Table 3 lists where the implementation departs from the original ODD specification and why.

3.1 Entities and state variables

The model contains one **orchestrator**, N **workers** (5 to 100), and optionally one **monitor**. The orchestrator holds a believed progress value, its estimate of mean task completion. Each worker holds its true progress on its current subtask, the progress it reports, the resources it received this tick, a trust score assigned by the orchestrator, a logging rate, a resource pressure and an alive flag. The monitor holds an anomaly score. A global resource pool represents the shared context budget. One tick stands for one request–response cycle of an agent system; runs last 200 ticks.

3.2 Process overview

Each tick runs eight steps in a fixed order, with synchronous updating:

1. **Distribute resources.** The pool is split among alive workers, weighted by trust.
2. **Execute task.** Each worker advances its true progress in proportion to its resources. A finished worker receives a new subtask.
3. **Report.** Each worker computes a reported progress, with noise that grows with pressure, and attempts to deliver it.
4. **Update world model.** The orchestrator updates its belief from the reports it actually received and updates trust scores.
5. **Monitor.** The monitor, if present, samples workers and updates its anomaly score.
6. **Cascade failures.** Workers whose resources fall below the survival threshold die; in the hierarchical topology, dependents of a dead broker can die too.
7. **Metrics.** The five misalignment metrics and the composite score are computed.
8. **Display.** The interface is updated.

3.3 Key submodels

Resources. The pool is provisioned per worker slot, and the share of a dead worker is not redistributed. With scarcity **s** and trust **t_i**:

```
pool = floor(N × 10 × (1 − 0.8 s))
base = pool / N
held_i = min(base × (0.7 + 0.6 t_i), 1.5 × base)
ideal = 10 × N / n_alive
pressure_i = max(0, 1 − held_i / ideal)
```

Survivors inherit the subtasks of dead workers, so their ideal share grows as the population shrinks. This creates the model's positive feedback: deaths raise pressure on survivors.

Work and reporting. True progress grows by held_i / 80 per tick, so a fully resourced worker finishes a subtask in about 8 ticks. The reported progress is the true progress minus a noise term, pressure × 0.3 × U(0,1). When the optional strategic-bias rule is on and pressure exceeds 0.5, a bias drawn from U(−0.15, +0.05) is added, a slight tendency to under-report. The logging rate is max(0.05, 1 − 0.8 × pressure × U(0,1)), and a report reaches the orchestrator with probability equal to that rate. In the ring topology a

failed report can be relayed by the next worker; in the hierarchy, outer-tier workers report through an inner-tier broker, so a report must survive two hops.

Orchestrator and trust. The orchestrator cannot observe true progress. Its belief is the mean of the last report it received from each alive worker, some of which may be stale. Trust is updated from agreement with the group: $t_i \leftarrow 0.9\, t_i + 0.1\,(1 - |\text{reported}_i - \text{belief}|)$.

Monitor. The monitor samples $\max(1, \text{round}(0.2 \times n_{alive} \times (1 - s)))$ workers per tick, because monitoring also consumes context budget, and updates an exponentially smoothed mean absolute gap between true and reported progress. Its alarm threshold is 0.2.

Failure. A worker dies when its resources fall below the survival threshold (3 tokens by default). In the hierarchy, each dependent of a dead broker dies with probability 0.5.

3.4 Metrics

Table 2. Misalignment metrics, computed each tick over alive workers.

Metric	Definition	Weight in composite
IAI	Information Asymmetry Index: mean |true − reported progress|	0.25
Gini	Gini coefficient of resources held	0.20
TCD	Task Completion Drift: |orchestrator belief − mean true progress|	0.25
OER	Oversight Erosion Rate: 1 − mean logging rate	0.15
CFP	Cascade Failure Proportion: workers dead / N	0.15

The composite score is the weighted sum. A run is in WARNING when the composite reaches 0.28 and CRITICAL at 0.45; it is also labelled CRITICAL when every worker is dead. In that case the four process metrics are set to 1 and the composite keeps its last computed value.

3.5 Implementation and departures from the original ODD

The model is implemented in NetLogo 7.0.4 (Wilensky, 1999). Testing the first version against the ODD revealed several places where the code did not do what the specification intended, or where the specification itself was inconsistent. Version 2 corrects them (Table 3). Porting to NetLogo 7 also required three code fixes: invalid one-line reporter definitions, a local variable that clashed with the N slider because NetLogo ignores case, and a link-colouring routine that would have raised a runtime error on the first tick.

Table 3. Differences between the original ODD specification and the implemented model.

Element	Original ODD	Implemented (v2) and reason
Resource pressure	1 − held / (1.5 × base)	Measured against the ideal share. In v1, scarcity cancelled out and pressure stayed near 0.14.
Survival threshold	Fixed at 1.0	Slider, default 3. No worker could fall below 1.0, so failure never occurred.
Base share	pool / alive workers	pool / N: a dead worker's share is not redistributed, and survivors inherit its subtasks.
Trust update	Consistency with true progress	Agreement with the group's reports. The orchestrator cannot observe true progress, as the ODD itself notes.
Report delivery	Logging recorded but reports always used	Reports arrive with probability = logging rate, so logging failure causes stale beliefs; ring relay and two-hop hierarchy added.
Monitor coverage	20% of workers	20% × (1 − scarcity), at least one worker.
Subtasks	One subtask per worker	A finished worker receives a new subtask, so dynamics continue after about tick 8.

Element	Original ODD	Implemented (v2) and reason
Cascade trigger	Bernoulli(pressure × 0.08) per tick	Deterministic threshold on resources held.
Strategic bias	Always active above pressure 0.5	Switch added (strategic-bias?) to allow an ablation.

4 Experimental design

All experiments were run with NetLogo BehaviorSpace, 200 ticks per run, the monitor enabled, and metrics recorded at the final tick except in experiment 5, which records every tick. Table 4 summarises the design; 4,080 runs were performed in total.

Table 4. Experiments.

Experiment	Varied	Fixed	Reps	Runs
1 Main sweep	scarcity 0.10–0.95 (step 0.05); topology star, ring, hierarchical	N 50, threshold 3, bias on	30	1,620
2 Threshold	survival threshold 2.5, 3, 3.5; scarcity 0.80–0.95 (step 0.025); topology	N 50, bias on	20	1,260
3 Population	N 10, 25, 50, 100; scarcity 0.5, 0.7, 0.85, 0.9; star, hierarchical	threshold 3, bias on	20	640
4 Bias ablation	scarcity 0.1–0.9 (step 0.1); strategic bias on, off	star, N 50, threshold 3	30	540
5 Trajectories	scarcity 0.3, 0.6, 0.85, 0.95 (every tick)	star, N 50, threshold 3	5	20

Two additional reporters were recorded: the tick at which a run stopped, which measures time to collapse, and the first tick at which WARNING and CRITICAL were reached. A run is counted as **collapsed** when no worker is alive at the end.

Verification. Before the campaign, final metric values for N = 50 in the star topology were compared with an independent Python re-implementation of the same rules at scarcity 0.5, 0.8, 0.9 and 0.95; the two agreed to within about 0.01. Results were exported as CSV, imported into Excel with explicit decimal types, and summarised with formulas over the raw tables; the summaries were cross-checked against an independent computation in Python. Group differences were tested with χ^2 and Fisher exact tests for collapse counts, Mann–Whitney tests for run length, and Welch t-tests for the ablation.

5 Results

5.1 Slow drift, then sudden collapse

Figure 1 shows the main result. Up to scarcity 0.45 the system is essentially perfect. From 0.5 onwards all process metrics rise steadily: in the star topology IAI grows from 0.034 at scarcity 0.5 to 0.148 at 0.9, and the composite from 0.033 to 0.117. No worker dies and no run reaches WARNING anywhere below scarcity 0.95.

Between 0.90 and 0.95 the behaviour changes qualitatively. At 0.95 every run reaches WARNING, workers die in sequence, and many runs end with no worker alive. The mean composite jumps to 0.33–0.34. The composite itself crosses the CRITICAL line (0.45) in only 10% of star and 17% of hierarchical runs, and never in the ring; most CRITICAL labels at the end of a run reflect total collapse rather than a high score. The system therefore gives little graded warning: the slow drift before the edge is real but stays far below the alert thresholds.

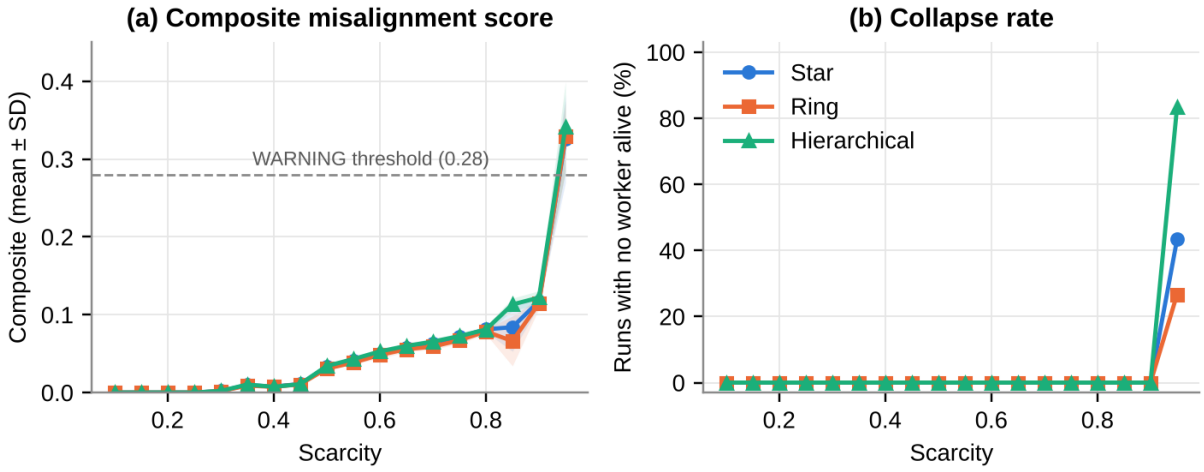


Figure 1. Experiment 1 (30 runs per point). (a) Mean composite score with ±1 SD band; the dashed line is the WARNING threshold. (b) Share of runs ending with no worker alive.

5.2 Topology matters only at the edge

Below scarcity 0.95 the three topologies are almost indistinguishable. Differences in the composite are statistically detectable (Kruskal–Wallis $p = 0.002$ at scarcity 0.9) but small, at most 0.008. At 0.95 they diverge sharply (Table 5). The hierarchy collapses in 25 of 30 runs, against 13 for the star and 8 for the ring ($\chi^2 = 20.4$, $p < 0.001$). The hierarchy differs significantly from both the star (Fisher $p = 0.003$) and the ring ($p < 0.001$), while star and ring do not differ significantly ($p = 0.28$). Hierarchical runs also fail sooner: they stop after 117 ticks on average, against about 180 for the other two (Mann–Whitney $p < 0.001$ against the star), and reach WARNING around tick 50 rather than tick 87–92.

Table 5. Experiment 1 at scarcity 0.95 (30 runs per topology).

Measure	Star	Ring	Hierarchical
Runs collapsed	13 (43%)	8 (27%)	25 (83%)
Mean ticks before stop	181	186	117
Mean first tick in WARNING	87	92	50
Mean composite	0.326	0.329	0.342
Runs with composite ≥ 0.45	10%	0%	17%

The mechanism is the secondary cascade: when an inner-tier broker dies, each worker that depends on it dies with probability 0.5, so a single local failure removes several workers at once. The ring's relay, by contrast, gives a failed report a second chance, which keeps the orchestrator better informed.

5.3 The tipping point is set by the survival threshold

Experiment 2 moves the survival threshold (Figure 2). With a threshold of 2.5 no run collapses anywhere up to scarcity 0.95. With 3, collapse appears only at 0.95. With 3.5, it starts at 0.90 (85–100% of runs, depending on topology) and is total from 0.925.

This boundary can be derived from the allocation rule. A worker survives while $\text{base} \times (0.7 + 0.6\,t) \geq \text{threshold}$. At scarcity 0.95 the base share is 2.4 tokens, so with a threshold of 3 a worker survives only while its trust stays above about 0.92; at 0.90 the base share is 2.8 and trust need only exceed about 0.62. Near that boundary, small losses of trust caused by noisy reports push workers below the threshold, deaths raise the pressure on survivors, and the cascade feeds itself. **Where** the system tips is therefore a direct consequence of parameter choices; **how** it tips, through a self-reinforcing cascade whose severity depends on topology, is the emergent part.

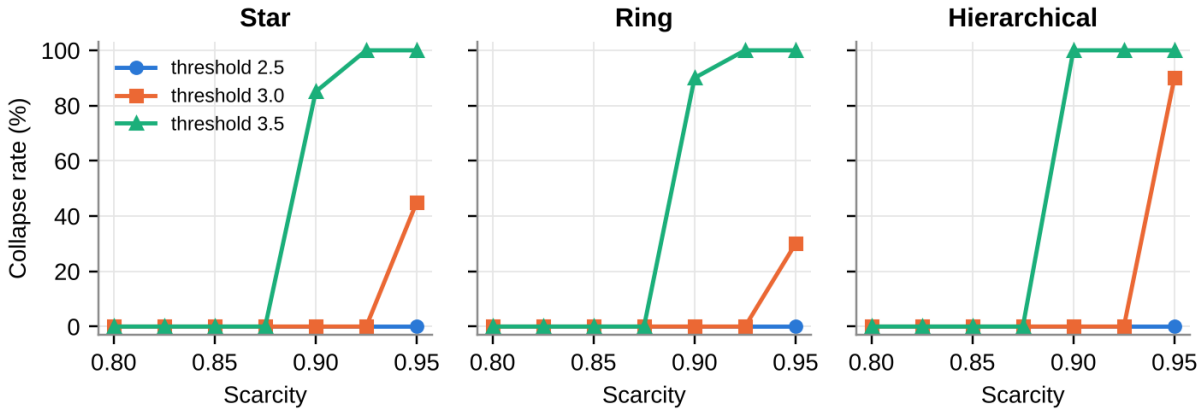


Figure 2. Experiment 2: share of collapsed runs by survival threshold and topology (20 runs per point).

5.4 Population size has little effect

In experiment 3 the composite is almost identical for 10, 25, 50 and 100 workers at every scarcity level tested; for example, at scarcity 0.7 in the star topology it ranges from 0.062 to 0.063. No run collapses, since the highest scarcity tested is 0.9. Larger populations only make outcomes more predictable: the run-to-run standard deviation of the composite at scarcity 0.7 falls from 0.008 with 10 workers to 0.003 with 100. This is expected, because resources are allocated per worker slot, so scarcity means the same thing for every population size.

5.5 Misalignment emerges without the biased rule

The strategic-bias rule is the only rule that pushes reports in a particular direction. Experiment 4 switches it off (Figure 3). Up to scarcity 0.7 the information gap is the same with and without it (IAI 0.066 without, 0.064 with; Welch $p = 0.11$). The rule only contributes above 0.8, where mean pressure exceeds 0.5 and the rule becomes active: it then accounts for 21% of IAI at scarcity 0.8 and 33% at 0.9 ($p < 0.001$ in both cases).

Most of the information gap therefore comes from pressure-driven noise, failed delivery of reports and stale beliefs, not from a tendency to misreport. One caveat applies: the noise term is itself a modelling choice that grows with pressure. The accurate claim is that no deceptive or directional rule is needed for the gap to appear, not that no rule produces it.

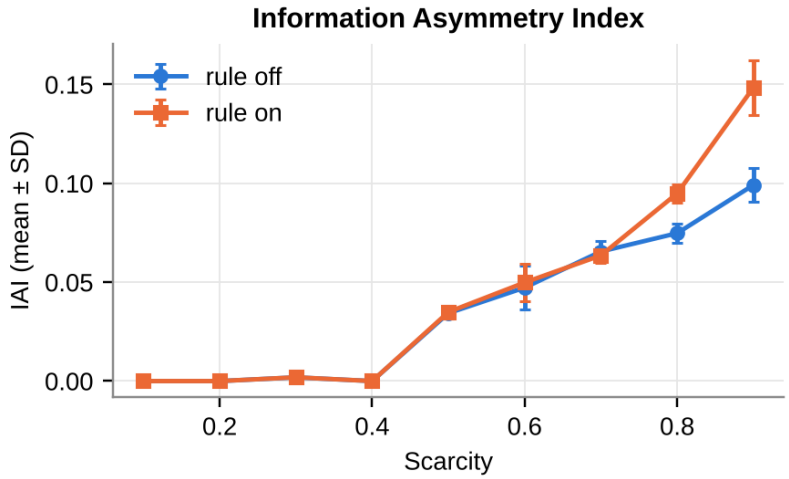


Figure 3. Experiment 4: IAI with the strategic-bias rule off and on (mean ± SD, 30 runs per point).

5.6 The monitor sees the problem but does not raise the alarm

Across all experiment 1 runs below scarcity 0.95, the monitor's anomaly score is strongly correlated with the true IAI (r = 0.92), so in principle it measures the right thing. In practice its alarm threshold (0.2) is almost never crossed: only 3 of 1,530 runs end above it below scarcity 0.95, and about a third of runs at 0.95, while the system is collapsing. At 0.95 the monitor samples a single worker per tick, because its own coverage shrinks with scarcity. The monitor is thus least effective exactly when it is most needed.

5.7 Dynamics over time and a measurement artefact

Experiment 5 shows how runs evolve (Figure 4). Up to scarcity 0.85, the composite settles to a stable level within about ten ticks. At 0.95, it keeps rising as workers die: the first deaths occur around tick 20, half the workers are gone by about tick 50, and on average about 3% remain at tick 200.

Figure 4(a) also reveals an artefact. Workers receive similar resources and therefore finish their subtasks at nearly the same tick; when they do, true and reported progress both reset to zero together, and IAI and TCD collapse for a few ticks before recovering. A run that ends during such a dip records an artificially low value. This explains the irregular points in the final-tick results, such as the dip at scarcity 0.85 in Figure 1, at 0.925 in experiment 2, and for N = 100 in experiment 3. Averaging the metrics over the last ticks of a run would remove it.

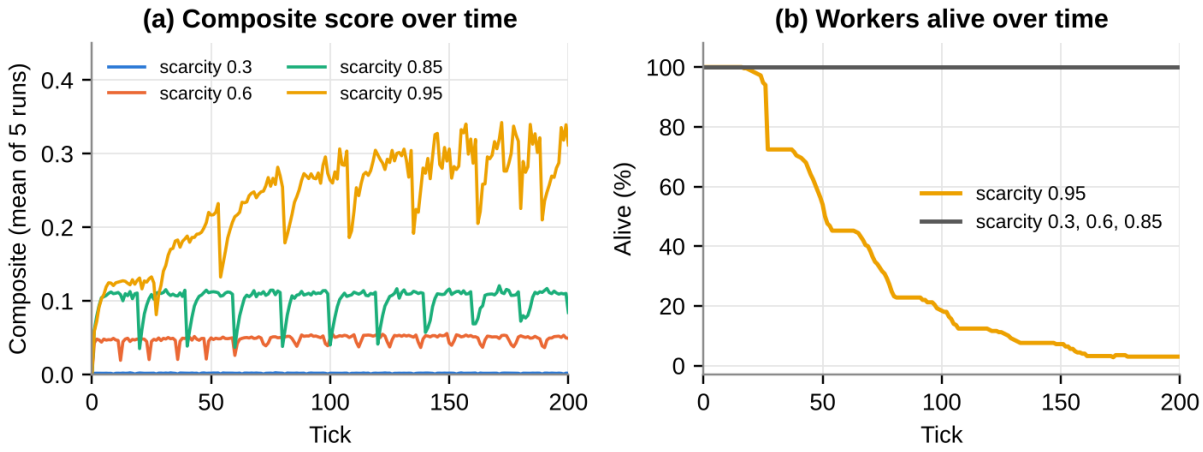


Figure 4. Experiment 5 (mean of 5 runs per scarcity level). (a) Composite score over time; the regular dips occur when workers finish their subtasks simultaneously. (b) Share of workers alive; at scarcity 0.3, 0.6 and 0.85 all workers survive.

5.8 Evaluation against the patterns predicted by the ODD

The ODD protocol listed five patterns as criteria for model validity. Table 6 compares them with the results.

Table 6. Predicted patterns and outcomes.

Pattern (ODD §1.2)	Prediction	Outcome	Evidence
Phase transition in IAI	Step-like rise of IAI beyond scarcity 0.5–0.6	Partly	IAI rises smoothly from 0.5; the abrupt shift occurs at 0.90–0.95 and concerns survival, not IAI alone.
Matthew effect	Trust concentrates resources; right-skewed Gini	Not reproduced	Gini stays below 0.02 before collapse: all workers face similar pressure, so trust stays uniform.
Topology and cascades	Hierarchy more fragile than star	Supported	Collapse at 0.95: 83% vs 43% (p = 0.003).
Monitor degradation	Monitor less effective under scarcity	Supported, by design	Alarm almost never fires; sample falls to one worker. Built into the coverage rule rather than emergent.
Reporting bias without deception	Reports diverge from truth without a deception rule	Supported	Ablation: identical IAI up to 0.7 with the rule off.

6 Discussion

6.1 What the model shows

Within its assumptions, MESA shows that a system of honest agents can drift and then fail without any agent being designed to cheat. Three features of that failure stand out. First, the decline is quiet: before the edge, every metric stays well below the alert thresholds, so the system looks healthy until it is not. Second, the first casualty is oversight: reporting quality and logging degrade before any task fails, so the coordinator's view of reality erodes first. Third, structure matters most when it matters least visibly: topology makes almost no difference in normal conditions but decides how badly the system fails at the edge.

The results also correct the original expectations. The predicted step in IAI at moderate scarcity did not appear, and the predicted Matthew effect did not emerge. These negative results are informative: a trust rule that rewards agreement with the group does not concentrate resources when all agents are under the same pressure. Inequality would require heterogeneity, such as agents with different capabilities or tasks of different sizes.

6.2 Implications for running AI agents on a personal computer

MESA is a stylised model, so its numbers do not transfer to real systems; what transfers is a set of risks worth watching for. Several agents running on one laptop share memory, processor time, context windows and any API budget or rate limit. Running more agents, or giving them larger tasks, gives each a thinner slice, which is the situation the model simulates. Under that analogy, the results suggest five practical precautions:

- **Keep headroom.** Performance may hold up and then fail abruptly. Running agents near the limit of the machine or the budget is risky because the limit is not visible until it is crossed.
- **Verify outputs, not reports.** Under load, an agent's statement that a task is done is the first thing to become unreliable. Check the actual result, such as a file that exists or tests that pass.
- **Prefer flat structures.** Layers of sub-coordinators add points of failure whose loss takes their dependents down. One coordinator with a few workers is more robust.
- **Do not equate honesty with reliability.** The problems arose without any deceptive agent. Well-behaved agents can still give a collectively wrong picture when overloaded.
- **Give monitoring its own resources.** A checker that shares the constrained budget checks least when it matters most. Logging and verification should have a guaranteed share.

These are hypotheses suggested by the simulation, not established findings about real agents. Section 8 proposes how to test them.

7 Limitations

- **Stylised agents.** Workers follow fixed rules with uniform noise. Real language-model agents reason, use tools, retry and correct each other, none of which is modelled.
- **Parameter-dependent tipping point.** The location of the collapse follows directly from the survival threshold and the allocation formula (Section 5.3). Results should be read as a sensitivity analysis, not as a universal critical scarcity.
- **Composite calibration.** The composite crosses CRITICAL only rarely; CRITICAL mostly reflects total collapse, which the model labels directly. In collapsed runs the composite is frozen at its last value and the process metrics are set to 1.
- **Final-tick snapshots.** Synchronised task completion produces periodic dips in IAI and TCD, which make some final-tick values unrepresentative (Section 5.7).

- **Designed-in mechanisms.** Monitor degradation and pressure-driven noise are built into the rules. The model tests their system-level consequences, not their existence.
- **Homogeneous agents.** All workers are identical, which likely explains the absence of a Matthew effect.

8 Future work

8.1 Validation with real agents on a consumer laptop

The most informative next step is to reproduce the experiment with real AI agents on a common laptop with 4 GB of RAM. A modest machine is an advantage here, because it provides genuine scarcity. Two designs are possible, and they differ in what scarcity means.

Option A: models running locally. With 4 GB of RAM only very small language models can run, roughly 0.5 to 1.5 billion parameters in quantised form, served through a local runtime such as Ollama or llama.cpp. Several agents would typically share one loaded model and take turns. Scarcity is then physical (memory, processor time, context length), which is the closest analogue to MESA. Because small models make errors even without pressure, a single-agent baseline is needed to separate model weakness from system overload.

The baseline is obtained by running the full task set with the single agent alone on the laptop, with no other agent competing for memory, processor time or loaded model. Its error rate and its rate of false completion claims define what the small model gets wrong on its own. In the pressure runs, with two, four or eight agents sharing the machine, or with a reduced context length, only the excess over this baseline is attributed to scarcity; for example, if the baseline fails 20% of tasks and eight concurrent agents fail 45%, the system-level effect is 25 points. The baseline must use the same model, quantisation, prompts and runtime settings as the pressure runs, so that any difference comes from scarcity and not from the setup.

Option B: a production orchestration platform. Mistral Workflows runs in a hybrid mode in which the orchestrator is hosted by Mistral while the worker code runs in the user's own environment, including a laptop for local development (Mistral AI, 2026). The 4 GB laptop then hosts the workers, which call Mistral models through the API, and scarcity is imposed explicitly: the number of concurrent workers on the machine, a token budget per worker, a maximum context length, or a rate limit. This is easier to control precisely and is the direct test of the analogy in Table 1, at a small monetary cost.

Option B also offers ground truth that the simulation can only assume. The platform's event history records every completed step independently of what the agents report, so IAI and TCD can be measured by comparing agent statements with the recorded history, and oversight erosion by the share of steps that appear in the traces. It also adds a second question. A durable platform logs each step at infrastructure level and retries failed activities on its own, which are exactly the mechanisms that MESA shows eroding under pressure. Running the same tasks once through the platform and once as a plain agent loop without durable execution would test whether infrastructure-level logging and retries prevent the oversight erosion and collapse that the model predicts.

A baseline is needed here as well. Cloud models make fewer errors than small ones, but not none, so the task set is first run by a single worker with an unrestricted token budget and no concurrent load. Its error rate of false completion claims define what the model gets wrong on its own. In the pressure runs, only the excess over this baseline is attributed to scarcity; for example, if the baseline fails 5% of tasks and eight concurrent workers fail 20%, the system-level effect is 15 points. The baseline must use the same model, prompts and platform configuration as the pressure runs, so that any difference comes from scarcity and not from the setup.

In both designs the MESA metrics map onto measurable quantities (Table 7). Tasks should have answers that a script can check automatically, for example small coding exercises with unit tests, extraction from

documents with a known answer key, or file-processing jobs. Pressure is then increased step by step, with more agents or smaller budgets, in each of the three topologies.

Table 7. Mapping MESA quantities to a real-agent experiment.

MESA	Real-agent experiment
Scarcity	Number of agents sharing the machine, or the token / context budget per agent
True vs reported progress (IAI)	What an agent says it did vs what a script verifies it did (tests pass, file exists, answer correct)
Orchestrator belief vs reality (TCD)	The coordinator's final status report vs the verified results
Logging failure (OER)	Share of steps the agents record in the required log; in Option B, share of steps present in the traces
Collapse (CFP)	Agents crashing, timing out, running out of memory or abandoning the task
Topology	Coordinator with workers; chain; coordinator → sub-coordinators → workers

The test is whether real systems show the same two signatures: quality that drifts slowly and then falls off a cliff, and a coordinator's view that diverges from reality before the failure. Agreement would lend the simulation credibility; disagreement would show which of its assumptions is wrong. Option B is also the ecological-validity test proposed in the original ODD, carried out at laptop scale.

8.2 Model improvements

- Average metrics over the last 20 ticks, or record them at every step, to remove the snapshot artefact.
- Introduce heterogeneous workers and subtask sizes, which would desynchronise task completion and give the Matthew effect a chance to appear.
- Restore the stochastic cascade trigger of the original ODD and compare it with the deterministic threshold.
- Apply the statistical tests planned in the ODD: Kolmogorov–Smirnov tests for the transition and regression models for topology and population effects.
- Recalibrate or justify the composite weights and phase thresholds, for example against outcomes in the real-agent experiment.

9 Conclusion

MESA shows, in a controlled simulation, that individually aligned agents can produce collectively misaligned outcomes under resource scarcity. Oversight erodes first and quietly, the system then fails abruptly, and the structure of the agent team decides how badly. None of this requires a deceptive agent. The precise point of failure depends on modelling choices, and two of the five predicted patterns were not reproduced, which the paper reports as findings in their own right. The practical message for anyone running several AI agents on limited hardware is to keep headroom, verify results rather than trust reports, keep structures flat and protect the resources of whatever checks the work. Whether real agents behave this way is an empirical question, and the validation experiment proposed here is designed to answer it.

Acknowledgements

This work extends the ODD protocol written as the final assignment of the Agent-Based Modelling course taught by Georgiy Bobashev in the MSc Data Science & AI programme at DSTI School of Engineering. The implementation, experiments and this paper were carried out afterwards as a personal project.

Use of AI assistance. Claude (Anthropic) was used to port the model to NetLogo 7, review and fix code, configure the BehaviorSpace experiments, check the imported data and build the analysis workbook,

compute the statistical tests and figures, and draft the text of this paper. The author designed the model and the research question, ran the experiments, and reviewed the results and the text.

Data and code

The model (MESA_v2.nlogox, NetLogo 7.0.4), the experiment definitions (BehaviorSpace_setup.md and MESA_experiments_2-5.xml), the raw BehaviorSpace output and the analysis workbook (MESA_analysis.xlsx) accompany this paper and are available from the author.

References

Barabási, A.-L. (2016). *Network Science*. Cambridge University Press.

Chan, A. et al. (2024). Visibility into AI agents. In *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency (FAccT 2024)*.

Gabriel, I. (2020). Artificial intelligence, values, and alignment. *Minds and Machines*, 30, 411–437.

Grimm, V. et al. (2020). The ODD protocol for describing agent-based and other simulation models: a second update to improve clarity, replication, and structural realism. *Journal of Artificial Societies and Social Simulation*, 23(2), 7. https://doi.org/10.18564/jasss.4259

Lynch, A. et al. (2025). *Agentic Misalignment: How LLMs Could Be Insider Threats*. Anthropic Research.

Mistral AI (2026). *Mistral Workflows: Durable Agent Orchestration*. Public preview, 28 April 2026. Documentation: https://docs.mistral.ai/workflows

Park, J. S. et al. (2023). Generative agents: interactive simulacra of human behavior. In *Proceedings of UIST 2023*.

Scheffer, M. et al. (2009). Early-warning signals for critical transitions. *Nature*, 461, 53–59.

Shoham, Y. & Leyton-Brown, K. (2009). *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press.

Wilensky, U. (1999). *NetLogo*. Center for Connected Learning and Computer-Based Modeling, Northwestern University.